

Adaptive Attention Computation: Balancing Efficiency & Retrieval

Consolidated 2-Person Research & Testing Sprint Plan • Contract-First Execution

Target Horizon: 8.5 – 9.5 Hours	Team Structure: 2 Engineers (Research + Testing)	Hardware: Kaggle Dual Tesla T4 (16GB VRAM)
Precision/Framework: PyTorch 2.11 (FP32/FP16)	Subject Model: Qwen/Qwen2.5-0.5B	Primary Deliverables: Kernels, CSVs, 7-Slide Deck

1. Executive Context & Core Research Pivot

Empirical benchmarks on Tesla T4 established that kernelized linear attention achieves **164.4× speedup at N = 65,536 (9.74 ms vs 1,601.74 ms for SDPA)** and constant internal state memory (64 KB/head), but causes catastrophic **retrieval collapse (14.2% accuracy at N = 1,024 vs 100% for Softmax)** due to key collisions in fixed-capacity buffers. This sprint restructures the original 6-person project into an agile **2-person sub-group** to execute the pivot to **Adaptive Attention Computation**: dynamically allocating feature capacity based on an analytical error certificate.

2. Sub-Group Role Division & Consolidated Responsibilities

Dimension	Person 1: Research Lead (Theory & Architecture)	Person 2: Testing Lead (Compute & Validation)
Merged Roles	Merges Person 1 (Theory) + Person 4 (Model Patch) + Person 6 (Deck & Synthesis)	Merges Person 2 (Compute) + Person 3 (Retrieval) + Person 5 (SOTA Baselines)
Core Mission	Formulate mathematical bounds, code adaptive-rank kernel algorithms, monkey-patch Qwen2.5-0.5B, and author presentation narratives.	Engineer T4 profiling harnesses, run synthetic Passkey retrieval suites, benchmark Gated DeltaNet baselines, and audit statistical non-inferiority.
Key Deliverables	- adaptive_kernel.py (Dynamic rank logic) - patch_qwen.py (Forward monkey-patch) - Master 7-Slide Deck + Executive Summary	01_T4_Efficiency.ipynb (Latency/VRAM curves) 02_Passkey_Suite.ipynb (Retrieval heatmaps) 03_DeltaNet_T4.ipynb + Consolidated CSVs
Primary Metric	Error bound adherence: $\epsilon_r(X) \leq \delta Z_i / [N(2 V_{\max} + \delta)]$	Empirical scaling exponent ($\alpha \approx 1.0$) & Bootstrap 95% CI ($\Delta > -0.05$)

3. Contract-First Decoupling (Hour 0 Interface Lock)

To eliminate idle waiting and cross-blocking, interfaces are frozen at Hour 0.0. The Testing Lead develops evaluation harnesses against a frozen mock kernel signature, while the Research Lead builds against an immutable tensor contract.

Frozen Kernel Signature (adaptive_kernel.py):

```
def adaptive_rank_attention(Q, K, V, delta=0.05, r_candidates=(64, 256, 1024), is_causal=False) -> torch.Tensor
• Inputs: Q, K: (B, H, N, d_k); V: (B, H, N, d_v) | Output: (B, H, N, d_v)
• Mock Kernel Protocol (Hours 1-6): Returns linear_attention(Q, K, V) with mock rank logging so test suites run immediately.
```

Standardized Benchmark CSV Schema:

```
columns = ['timestamp', 'sequence_length_N', 'method', 'metric_name', 'metric_value', 'peak_vram_mb', 'status']
```

4. Master Hour-by-Hour Execution Schedule & Synchronization Checkpoints

The 9.5-hour sprint is divided into 6 synchronized phases. Four mandatory synchronization checkpoints guarantee that both tracks remain perfectly aligned, culminating in a unified code freeze at Hour 7.5.

Window	Person 1: Research Lead (Theory & Architecture)	Person 2: Testing Lead (Compute & Validation)	Sync Checkpoint & Milestone
0.0 – 1.0 h <i>Phase 1</i>	- Formulate adaptive rank error bound: $\epsilon_r \leq \delta \sum_i$ / $[N(2V_{\text{max}}+\delta)]$ - Publish MockAdaptiveKernel stub - Outline 7-slide presentation structure	- Setup Kaggle T4 dual environment harness - Lock PyTorch SDPA memory-efficient flags - Implement synthetic Passkey injector - Freeze CSV evaluation schema	SYNC 1 (Hr 1.0) Interface Freeze Sign-off on mock kernel & CSV contracts.
1.0 – 3.5 h <i>Phase 2</i>	- Implement exact output error estimator in adaptive_kernel.py - Code rank selector ($r \in \{64, 256, 1024\}$) - Validate 3-token toy calculation ($[4.8, 5.2]$) - Setup Qwen2.5-0.5B patch harness	- Benchmark Softmax vs SDPA (N=64 to 32K) - Run Passkey baseline (verify 100% acc) - Implement PyTorch Gated DeltaNet layer - Build automated chart rendering script	SYNC 2 (Hr 3.5) Baseline Check Verify Softmax sanity check & DeltaNet layer.
3.5 – 6.0 h <i>Phase 3</i>	- Add FP32 accumulator stability protections - Code causal prefix-sum state logic - Test forward monkey-patch on Qwen2.5 - Draft Slides 1, 2, and 5 (Theory/Gap)	- Sweep Linear (ReLU+1) latency/VRAM to 64K - Run Passkey sweep on Linear & FAVOR+ - Profile DeltaNet throughput vs Linear on T4	SYNC 3 (Hr 6.0) Mid-Sprint Review Inspect scaling (9.7ms) & retrieval curves (14%).
6.0 – 7.5 h <i>Phase 4</i>	- Finalize production adaptive_kernel.py - Profile prefill latency on patched Qwen2.5 - Draft Slides 6 & 7 (Architecture & Roadmap)	- Ingest real adaptive_kernel.py into suite - Run full retrieval & latency evaluations - Compute bootstrap 95% CI non-inferiority - Export all final publication charts	MILESTONE (Hr 7.5) CODE FREEZE All data & kernels locked. No code edits.
7.5 – 8.5 h <i>Phase 5</i>	- Ingest final charts & tables into 7-slide deck - Write executive summary report & notes	- Audit CSV metrics against notebook logs - Verify Kaggle reproduction script executes	SYNC 4 (Hr 8.5) Deck Audit Verify all numbers match CSV logs.
8.5 – 9.5 h <i>Phase 6</i>	- Lead 15-minute presentation dry-run - Rehearse theory and math Q&A; defense	- Present empirical benchmarks during dry-run - Package deliverable archive & repository	FINAL SIGN-OFF Hr 9.5 Deliverables submitted.

SPRINT EXECUTION LIFECYCLE:

Hr 0.0: Contract Freeze → Hr 1.0–3.5: Independent Builds → Hr 3.5–6.0: Sweeps & Baselines → Hr 7.5: Code Freeze → Hr 8.5: Deck Audit → Hr 9.5: Sign-Off

5. RACI Responsibility Matrix

Workstream / Deliverable	Research Lead	Testing Lead	Sign-Off Lead
Mathematical Bounds & Error Certificate Derivation	Responsible (R)	Consulted (C)	Research Lead
Standalone Adaptive Kernel Implementation (adaptive_kernel.py)	Responsible (R)	Consulted (C)	Research Lead
Kaggle T4 Profiling Environment & OOM Guardrails	Consulted (C)	Responsible (R)	Testing Lead
Latency & Peak VRAM Profiling (N = 64 to 65,536)	Informed (I)	Responsible (R)	Testing Lead
Passkey Retrieval Benchmark Suite & Distractor Testing	Consulted (C)	Responsible (R)	Testing Lead
SOTA Gated DeltaNet PyTorch Baseline Layer	Consulted (C)	Responsible (R)	Testing Lead
Qwen2.5-0.5B Attention Monkey-Patching & Prefill Benchmark	Responsible (R)	Consulted (C)	Research Lead
Statistical Non-Inferiority Audit (Bootstrap 95% CI)	Consulted (C)	Responsible (R)	Testing Lead
Master 7-Slide Deck Synthesis & Executive Summary	Responsible (R)	Consulted (C)	Research Lead
End-to-End Reproduction Scripts & Deliverables Verification	Consulted (C)	Responsible (R)	Testing Lead

6. Operational Guardrails & Acceptance Decision Standards

Rule / Criterion	Specification & Enforcement Mechanism	Threshold / Action
FlashAttention-2 Workaround	Tesla T4 (SM 7.5 Turing) cannot run FlashAttention-2. Enforce PyTorch SDPA memory-efficient kernel.	mem_efficient=True; flash=False
Batch Size Guardrail	To prevent GPU OOM, enforce batch size B = 1 for all sequences where $N \geq 8,192$.	Mandatory at $N \geq 8K$
Speed Acceptance Target	Median latency speedup relative to naive softmax over the target sequence range (N = 1K to 64K).	$R^* \geq 2.0\times$ (Pass)
Quality Non-Inferiority	Paired retrieval accuracy difference $\Delta = \text{acc_method} - \text{acc_softmax}$ across shared random tasks.	Bootstrap 95% CI > -0.05

7. Sprint Deliverables Sign-Off Checklist

[]	1. Adaptive Kernel Codebase: adaptive_kernel.py with unit tests, FP32 accumulators, and causal prefix sums.
[]	2. Model Monkey-Patch: patch_qwen.py verifying forward passes and prefill throughput on Qwen2.5-0.5B.
[]	3. Kaggle T4 Benchmark Notebooks: 01_T4_Efficiency.ipynb, 02_Passkey_Suite.ipynb, and 03_DeltaNet_T4.ipynb.
[]	4. Consolidated Benchmark CSVs: efficiency_raw.csv, retrieval_raw.csv, and noninferiority.csv.
[]	5. Master 7-Slide Deck: Fully assembled slide deck with embedded charts, speaker notes, and executive summary.